Home Exam: Document Processing Pipeline Architecture

Background

Your company processes documents to extract and classify entities. The pipeline has two main stages:

Stage	Purpose
Extraction	Scans documents and extracts **tokens** (raw text snippets that might be entities: company names, people, addresses, dates, etc.)
Classification	Takes each token and classifies it into a category: `COMPANY`, `PERSON`, `ADDRESS`, `DATE`, `UNKNOWN`, etc.

Example Flow:

```
Document: "John Smith works at Acme Corp, located at 123 Main St."

NLP Extraction → Tokens:
  - "John Smith" (page 1, sentence 1)
  - "Acme Corp" (page 1, sentence 1)
  - "123 Main St" (page 1, sentence 1)

LLM Classification → Classified Tokens:
  - "John Smith" → PERSON
  - "Acme Corp" → COMPANY
  - "123 Main St" → ADDRESS
```

You are tasked with designing and building this system from scratch. There is no existing implementation — only the business needs described above.

Part 1: Architecture Design

Design an architecture for this document processing pipeline.

Requirements your design should address:

1. **Scalable processing stages** - The architecture should allow different stages of the pipeline (extraction, classification, API serving, etc.) to scale independently in the future. How you structure the code is up to you, but justify why your approach supports independent scaling.
2. **Support reruns** - Resume processing after partial failures; support full reprocessing of a document from scratch
3. **Track durations** - Measure extraction and classification processing time per document
4. **Support local development** - Engineers should be able to run and test locally

---

1.1 Technology Selection

Choose your technology stack and justify each decision:

Component	Options to Consider	Your Choice	Justification
Inter-component Communication	How pipeline stages coordinate (your choice of approach)	?	?
Data Storage	SQL, NoSQL, document stores, key-value, blob + index, etc.	?	?
Local Development	Emulators, containers, in-memory mocks, etc.	?	?

1.2 Data Model

Design a data model for storing document processing state and tokens.

- Define your schema for:
  - **Document manifest** (processing state, progress, timestamps)
  - **Individual tokens** (the extracted and classified entities)
- Explain how your model supports:
  - Querying tokens by classification, document, or page
  - Tracking progress (e.g., "150 of 500 tokens classified")
  - Concurrent updates without conflicts

1.3 System Communication

Design how components communicate.

- Define the contracts between components
- Draw an architecture diagram showing:
  - Components and their responsibilities
  - Communication flows between components
  - Data store interactions

1.4 Rerun and Recovery

Your system must support two rerun modes:

Partial rerun (crash recovery):

> *Processing starts on a document. Extraction produces 100 tokens. After classifying 30 of them, the system crashes. On restart, the system should detect what was already processed and continue from where it left off — without re-extracting or re-classifying tokens that were already completed.*

Full rerun (reprocess from scratch):

> *A document needs to be reprocessed entirely — perhaps the source text changed or the previous run produced bad results. The system should be idempotent to previous runs.*

Your design should:

- Explain how the system tracks processing progress and knows where to resume
- Ensure a full rerun cleanly replaces previous data without leaving stale artifacts
- Define where the "source of truth" for processing state lives

1.5 Duration Tracking

Design how to track processing time:

- What timestamps should be recorded?
- When should each timestamp be set?
- How would you calculate "extraction took 5s, classification took 45s" for a document?

Part 2: Trade-offs and Edge Cases

2.1 Trade-off Analysis

Analyze the trade-offs in your design choices. For each major decision, explain:

- What alternatives did you consider
- Why did you choose your approach
- What you're giving up

2.2 Failure Scenarios

Explain how your design handles these failure scenarios:

- The classification component crashes after classifying a token but before persisting the result
- Extraction crashes mid-way through processing (some tokens written, others not)
- Database/storage is temporarily unavailable

Part 3: Working POC

Build a working proof-of-concept that demonstrates your architecture.

3.1 NLP and LLM Integration

NLP and LLM integrations may be **mocked**. You don't need to use a real NLP library or LLM API — but you must define clean interfaces for both and provide stub implementations.

Extraction interface should:

- Accept a document (plain text input is fine)
- Return extracted entities with:
  - The extracted text
  - Entity type from NLP (e.g., `PERSON`, `ORG`, `GPE`, `DATE`)
  - Position in document (sentence/paragraph number, character offset)

Classification interface should:

- Take a token (entity text + context)
- Return a classification and confidence/reasoning

Your mock implementations should return realistic-looking data so the rest of the system can be exercised end-to-end. For example, a simple rule-based stub or hardcoded responses per test document are fine.

Note: If you choose to use a real NLP library or LLM API, that's fine too, just provide your own API keys and include setup instructions so the evaluator can run your solution.

3.2 Core Features

Your POC must demonstrate:

Feature	Description
Per-token storage	Each token is stored individually
Rerun support	Partial reruns resume from last checkpoint; full reruns discard previous data cleanly
Progress tracking	Document status shows `processed_count / total_tokens`
Duration tracking	Timestamps for extraction start/end and classification start/end

3.3 Test Documents

Use realistic text documents for testing. You can use:

- News articles
- Company press releases
- Wikipedia excerpts
- Sample business documents

Provide at least 3 test documents in your repository with varying complexity:

- Small (5-10 entities)
- Medium (20-50 entities)
- Large (100+ entities)

3.4 Scenarios to Demonstrate

Provide a way to demonstrate these scenarios (test script, CLI commands, or instructions):

Scenario	What to demonstrate
Happy path	Process a document end-to-end, query results via API
Progress visibility	While processing, show real-time progress updates
Partial rerun	Start processing, kill it mid-way, restart. Show the system resumes from where it left off
Full rerun	Reprocess a previously completed document from scratch. Show previous data is fully replaced
Concurrent documents	Process 3 documents simultaneously
Query tokens	API to list tokens filtered by `document_id`, `classification`, or other fields

3.5 Local Setup

Provide a way to run everything locally. The evaluator should be able to:

```
# Start all services
./start.sh  # or docker-compose up, or make run, etc.

# Process a document
curl -X POST http://localhost:8080/process \
  -H "Content-Type: application/json" \
  -d '{"document_id": "doc-123", "text": "John Smith works at Acme Corp..."}'

# Check status
curl http://localhost:8080/documents/doc-123/status

# Query tokens
curl http://localhost:8080/documents/doc-123/tokens?classification=PERSON
```

3.6 Code Quality Expectations

Aspect	Expectation
Structure	Clean separation of concerns
Error handling	Graceful handling of failures (timeouts, rate limits, etc.)
Configuration	Environment-based config (API keys, service URLs)
Documentation	README with setup instructions and architecture overview
Testing	Integration tests demonstrating the scenarios

Deliverables

Deliverable	Format
Technology selection with justification	Table + explanation
Architecture design document	Markdown with diagrams
Data model schemas	Typed definitions or JSON Schema
Architecture diagram	Mermaid, draw.io, or similar
Communication contracts / API interfaces	Examples
Trade-off analysis	Explanation of decisions
Working POC	Git repository
Setup instructions	README.md
Test documents	At least 3 sample documents
Integration tests	Cover core scenarios (happy path, reruns, concurrent processing)

Demo script	Walkthrough or CLI commands demonstrating all scenarios
AI proficiency	Commit the prompts you've used to interact with the agent

---

Evaluation Criteria

Area	What we're looking for
Technology Choices	Reasonable selections with solid justification
Design Quality	Scalable, resilient, well-reasoned architecture
Rerun & Recovery	Sound strategy for partial and full reruns
NLP Integration	Clean interface design; extraction works correctly (mocked or real)
LLM Integration	Clean interface design; classification works correctly (mocked or real); if real — effective prompting, handles errors/rate limits, cost-conscious
Code Quality	Clean, readable, well-structured code
POC Completeness	All scenarios work as expected
Documentation	Clear setup instructions, architecture explanation
AI Proficiency	Committed prompts and evidence of deliberate, accountable AI usage